American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Mid-Term Project Report

Spring 2025-2026

Section:c

Group:11

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name:G.K.M Tamjid Ur Rahim ID:22-49771-3	All Tasks of the Project
2.	Name: ID:	
3.	Name: ID:	
4.	Name: ID:	

Dataset Description

Write a short description of your dataset, focusing on the features and target value.

The dataset used in this project is the Bank Marketing dataset from a Portuguese retail bank's direct marketing campaigns. It contains 45,211 client records and 17 attributes describing demographic information, financial status, and details about the last marketing contact. Numerical attributes include age, account balance, duration of the last call in seconds, number of contacts during the current campaign, number of days since the client was last contacted in a previous campaign, and the number of previous contacts. Categorical attributes capture the client's job type, marital status, education level, credit default status, housing loan status, personal loan status, contact communication type, last contact month, outcome of any previous campaign, and the target variable y, which indicates whether the client subscribed to a term deposit (yes or no).

Project Implementation Detail

Title of the task1

Description of the task1: which task you are going to solve here and how you will solve it.

In this task, I investigate missing values in the bank marketing dataset and clean the categorical attributes for further analysis. First, I check each variable for true NA values and then count how many times the level "unknown" appears in the categorical features, because "unknown" is used in this dataset to represent missing or unavailable information. After identifying these entries, I remove records with unknown education and impute unknown job values with the most frequent job category to create a cleaner dataset (bank_clean) for the next steps.

Code for task1

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)


factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")
bank[factor_cols] <- lapply(bank[factor_cols], factor)


na_counts <- colSums(is.na(bank))


factor_cols <- c("job", "marital", "education", "default",
```

```
            "housing", "loan", "contact", "month",
            "poutcome", "y")
unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))

bank_clean <- bank
bank_clean <- subset(bank_clean, education != "unknown")

job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[1]
bank_clean$job[bank_clean$job == "unknown"] <- job_mode

remaining_unknowns <- sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))
dim_bank_clean <- dim(bank_clean)
```

Sample output of your code for task1

```
> na_counts
      age       job   marital education   default   balance   housing      loan   contact       day     month  duration
        0         0         0         0         0         0         0         0         0         0         0         0
 campaign     pdays  previous  poutcome         y
        0         0         0         0         0
>
> # Check 'unknown' levels in categorical variables
> factor_cols <- c("job", "marital", "education", "default",
+                  "housing", "loan", "contact", "month",
+                  "poutcome", "y")
>
> unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
> unknown_counts
      job   marital education   default   housing      loan   contact     month  poutcome         y
      288         0      1857         0         0         0     13020         0     36959         0
> job_mode
[1] "blue-collar"
>
> bank_clean$job[bank_clean$job == "unknown"] <- job_mode
>
> # Check remaining unknowns
> sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))
      job   marital education   default   housing      loan   contact     month  poutcome         y
        0         0         0         0         0         0     12343         0     35425         0
>
> # New dimension after cleaning
> dim(bank_clean)
[1] 43354    17
```

Code description of task1: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

First, I converted the main categorical variables (job, marital, education, default, housing, loan, contact, month, poutcome, and y) to factors so that they behave correctly as categorical attributes in later analyses. I then calculated na_counts using colSums(is.na(bank)), which showed that none of the 17 variables contain true NA values. Next, I used unknown_counts to count the number of "unknown" entries in each categorical variable and found that education, job, contact, and poutcome contain a substantial number of "unknown" values, indicating missing or unspecified information in those fields. To clean the data, I created a copy called bank_clean, removed all records with unknown education, and replaced unknown job values with the most frequent job category (blue-collar) as a simple mode-based imputation. After

applying these steps, the cleaned dataset bank_clean has 43,354 records instead of 45,211, and there are no remaining unknown values in job or education, although contact and poutcome still contain some "unknown" levels that will be considered when interpreting later analyses.

Title of the task2

Description of the task2: which task you are going to solve here and how you will solve it.

In this task, I detect and handle outliers in key numerical variables of the bank marketing dataset, focusing on account balance and call duration. These attributes show very wide ranges and extreme values, which can distort descriptive statistics and later analyses. I use the interquartile range (IQR) method to identify observations below the lower fence or above the upper fence and then cap those extreme values at the corresponding thresholds to reduce the influence of outliers while keeping all records in the dataset.

Code for task2

```
numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

summary(bank_clean[, numeric_cols])

balance_q1 <- quantile(bank_clean$balance, 0.25)
balance_q3 <- quantile(bank_clean$balance, 0.75)
balance_iqr <- balance_q3 - balance_q1
balance_lower <- balance_q1 - 1.5 * balance_iqr
balance_upper <- balance_q3 + 1.5 * balance_iqr

duration_q1 <- quantile(bank_clean$duration, 0.25)
duration_q3 <- quantile(bank_clean$duration, 0.75)
duration_iqr <- duration_q3 - duration_q1
duration_lower <- duration_q1 - 1.5 * duration_iqr
duration_upper <- duration_q3 + 1.5 * duration_iqr

balance_outliers_before <- sum(bank_clean$balance < balance_lower | bank_clean$balance >
balance_upper)
duration_outliers_before <- sum(bank_clean$duration < duration_lower | bank_clean$duration
> duration_upper)
```

bank_clean2 <- bank_clean

bank_clean2$balance[bank_clean2$balance < balance_lower] <- balance_lower

bank_clean2$balance[bank_clean2$balance > balance_upper] <- balance_upper

bank_clean2$duration[bank_clean2$duration < duration_lower] <- duration_lower

bank_clean2$duration[bank_clean2$duration > duration_upper] <- duration_upper

balance_outliers_after <- sum(bank_clean2$balance < balance_lower | bank_clean2$balance > balance_upper)

duration_outliers_after <- sum(bank_clean2$duration < duration_lower | bank_clean2$duration > duration_upper)

summary(bank_clean2[, c("balance", "duration")])

Sample output of your code for task2

```
> numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")
>
> summary(bank_clean[, numeric_cols])
      age            balance           duration          campaign          pdays            previous
 Min.   :18.00   Min.   : -8019   Min.   :   0.0   Min.   : 1.00   Min.   : -1.00   Min.   :  0.0000
 1st Qu.:33.00   1st Qu.:    71   1st Qu.: 103.0   1st Qu.: 1.00   1st Qu.: -1.00   1st Qu.:  0.0000
 Median :39.00   Median :   443   Median : 180.0   Median : 2.00   Median : -1.00   Median :  0.0000
 Mean   :40.78   Mean   :  1355   Mean   : 258.2   Mean   : 2.76   Mean   : 40.34   Mean   :  0.5843
 3rd Qu.:48.00   3rd Qu.:  1415   3rd Qu.: 318.0   3rd Qu.: 3.00   3rd Qu.: -1.00   3rd Qu.:  0.0000
 Max.   :95.00   Max.   :102127   Max.   :4918.0   Max.   :58.00   Max.   :871.00   Max.   :275.0000
>
> balance_q1 <- quantile(bank_clean$balance, 0.25)
> balance_q3 <- quantile(bank_clean$balance, 0.75)
> balance_iqr <- balance_q3 - balance_q1
> balance_lower <- balance_q1 - 1.5 * balance_iqr
> balance_upper <- balance_q3 + 1.5 * balance_iqr
>
> duration_q1 <- quantile(bank_clean$duration, 0.25)
> duration_q3 <- quantile(bank_clean$duration, 0.75)
> duration_iqr <- duration_q3 - duration_q1
> duration_lower <- duration_q1 - 1.5 * duration_iqr
> duration_upper <- duration_q3 + 1.5 * duration_iqr
>
> balance_outliers_before <- sum(bank_clean$balance < balance_lower | bank_clean$balance > balance_upper)
> duration_outliers_before <- sum(bank_clean$duration < duration_lower | bank_clean$duration > duration_upper)
>
> bank_clean2 <- bank_clean
> bank_clean2$balance[bank_clean2$balance < balance_lower] <- balance_lower
> bank_clean2$balance[bank_clean2$balance > balance_upper] <- balance_upper
>
> bank_clean2$duration[bank_clean2$duration < duration_lower] <- duration_lower
> bank_clean2$duration[bank_clean2$duration > duration_upper] <- duration_upper
>
> balance_outliers_after <- sum(bank_clean2$balance < balance_lower | bank_clean2$balance > balance_upper)
> duration_outliers_after <- sum(bank_clean2$duration < duration_lower | bank_clean2$duration > duration_upper)
>
> summary(bank_clean2[, c("balance", "duration")])
    balance         duration
 Min.   :-1945   Min.   :  0.0
 1st Qu.:   71   1st Qu.:103.0
 Median :  443   Median :180.0
 Mean   :  926   Mean   :234.7
 3rd Qu.: 1415   3rd Qu.:318.0
 Max.   : 3431   Max.   :640.5
```

Code description of task2: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

First, I selected the main numerical variables (age, balance, duration, campaign, pdays, and previous) and examined their distributions using the summary function. The results showed very large ranges for balance and call duration, suggesting the presence of extreme values. I then applied the interquartile range (IQR) rule to both variables: for each one, I calculated the first quartile (Q1), third quartile (Q3), and IQR, and defined lower and upper fences as Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. Using these thresholds, I counted how many observations fell outside the fences, which represented outliers in balance and duration. To mitigate their influence without dropping records, I created a new dataset bank_clean2 and capped values below the lower fence and above the upper fence at the corresponding cutoff for each variable. After capping, the outlier counts for both balance and duration dropped to zero, and the summaries of these variables became less extreme while still preserving the overall shape of the data.

Title of the task3

Description of the task3: which task you are going to solve here and how you will solve it.

In this task, I create new features by converting variables between numeric and categorical forms and by normalizing a continuous attribute. I group the age variable into four age categories to obtain an interpretable categorical feature, convert the housing loan status from a yes/no factor into a numeric 0–1 indicator, and scale the call duration variable to a 0–1 range so that it is comparable with other features and less affected by its large absolute values.

Code for task3

```
bank_work <- bank_clean2


bank_work$age_group <- cut(
  bank_work$age,
  breaks = c(0, 30, 45, 60, Inf),
  labels = c("young", "middle", "mature", "senior"),
  right = TRUE
)


housing_num <- ifelse(bank_work$housing == "yes", 1, 0)
bank_work$housing_num <- housing_num


duration_min <- min(bank_work$duration)
duration_max <- max(bank_work$duration)
```

bank_work$duration_norm <- (bank_work$duration - duration_min) / (duration_max - duration_min)

str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration", "duration_norm")])

summary(bank_work[, c("age_group", "housing_num", "duration_norm")])

Sample output of your code for task3

```
> str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration", "duration_norm")])
'data.frame':   43354 obs. of  6 variables:
 $ age          : int  58 44 33 35 28 42 58 43 41 29 ...
 $ age_group    : Factor w/ 4 levels "young","middle",..: 3 2 2 2 1 2 3 2 2 1 ...
 $ housing      : Factor w/ 2 levels "no","yes": 2 2 2 2 2 2 2 2 2 2 ...
 $ housing_num  : num  1 1 1 1 1 1 1 1 1 1 ...
 $ duration     : num  261 151 76 139 217 380 50 55 222 137 ...
 $ duration_norm: num  0.407 0.236 0.119 0.217 0.339 ...
> summary(bank_work[, c("age_group", "housing_num", "duration_norm")])
  age_group      housing_num     duration_norm
 young : 6793   Min.   :0.0000   Min.   :0.0000
 middle:22979   1st Qu.:0.0000   1st Qu.:0.1608
 mature:12502   Median :1.0000   Median :0.2810
 senior: 1080   Mean   :0.5607   Mean   :0.3664
                3rd Qu.:1.0000   3rd Qu.:0.4965
                Max.   :1.0000   Max.   :1.0000
```

Code description of task3: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I first created an age_group variable using the cut function, which bins the numeric age into four categories (young, middle, mature, senior) based on fixed age intervals. This transforms a continuous attribute into a categorical one that is easier to interpret when comparing client segments. Next, I converted the housing loan variable from a factor with levels “no” and “yes” into a numeric indicator housing_num, where 0 represents no housing loan and 1 represents having a housing loan. Finally, I normalized the call duration by subtracting its minimum value and dividing by the range, producing duration_norm values between 0 and 1. The structure and summary outputs confirm that the new variables have the expected types and ranges, and these transformed features will be used in later descriptive statistics and comparisons.

Title of the task4

Description of the task4: which task you are going to solve here and how you will solve it.

In this task, I check whether the cleaned bank marketing dataset contains duplicate rows that could bias the analysis. Duplicate records may arise from repeated data collection or file merges and, if present, can overweight certain observations in descriptive statistics and later models. I use a built-in R function to identify duplicated rows and, if any exist, remove them to keep only unique client records.

Code for task4

bank_nodup <- bank_work

```
dup_flags <- duplicated(bank_nodup)
dup_count <- sum(dup_flags)

dim_before_dup <- dim(bank_nodup)

bank_nodup <- bank_nodup[!dup_flags, ]

dim_after_dup <- dim(bank_nodup)

dup_count
dim_before_dup
dim_after_dup
```

Sample output of your code for task4

```
> dup_count
[1] 0
> dim_before_dup
[1] 43354    20
> dim_after_dup
[1] 43354    20
```

Code description of task4: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I created a copy of the working dataset called bank_nodup and used the duplicated function to generate a logical vector dup_flags indicating which rows are exact duplicates of previous rows. Summing this vector gives dup_count, the total number of duplicate records. I recorded the dataset dimensions before and after removing duplicated rows by subsetting bank_nodup[!dup_flags,]. In my case, the number of duplicates was dup_count, and the dimensions before and after removal show that the dataset size did/not change, confirming that there were/no significant duplicate records after the earlier cleaning steps.

Title of the task5

Description of the task5: which task you are going to solve here and how you will solve it.

In this task, I detect and remove records with clearly invalid or uninformative values in key numerical variables of the bank marketing dataset. According to the data description, clients are adult individuals with ages roughly between 18 and 95, call duration is measured in seconds and should be positive, and the campaign variable counts the number of contacts in the current campaign. I treat ages outside the range 18–95, call durations equal to zero

seconds, and campaign values below 1 as invalid for the purpose of this analysis and filter out such records to obtain a more reliable dataset.

Code for task5

```
bank_final <- bank_nodup

invalid_age <- sum(bank_final$age < 18 | bank_final$age > 95)
invalid_duration <- sum(bank_final$duration <= 0)
invalid_campaign <- sum(bank_final$campaign < 1)

invalid_age
invalid_duration
invalid_campaign

bank_final <- subset(
  bank_final,
  age >= 18 & age <= 95 &
    duration > 0 &
    campaign >= 1
)

dim_bank_final <- dim(bank_final)
summary(bank_final[, c("age", "duration", "campaign")])
```

Sample output of your code for task5

```
> invalid_age
[1] 0
> invalid_duration
[1] 3
> invalid_campaign
[1] 0
> summary(bank_final[, c("age", "duration", "campaign")])
      age           duration        campaign
 Min.   :18.00   Min.   :  1.0   Min.   : 1.00
 1st Qu.:33.00   1st Qu.:103.0   1st Qu.: 1.00
 Median :39.00   Median :180.0   Median : 2.00
 Mean   :40.78   Mean   :234.7   Mean   : 2.76
 3rd Qu.:48.00   3rd Qu.:318.0   3rd Qu.: 3.00
 Max.   :95.00   Max.   :640.5   Max.   :58.00
```

Code description of task5: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I began with the de-duplicated dataset bank_nodup and copied it into bank_final for further cleaning. I then computed the counts of invalid values using logical conditions: ages below 18 or above 95, durations less than or equal to 0 seconds, and campaign counts less than 1. After checking how many such cases existed, I applied a subset filter requiring ages between 18 and 95, strictly positive call durations, and campaign values greater than or equal to 1. The resulting dimensions stored in dim_bank_final show that some records were removed, and the summaries of age, duration, and campaign confirm that the remaining data respect the defined validity rules. This cleaned dataset bank_final will be used in the balancing and splitting steps and for subsequent descriptive statistics.

Title of the task6

Description of the task6: which task you are going to solve here and how you will solve it.

In this task, I address the strong class imbalance in the target variable y and prepare the dataset for potential modeling by creating training and testing splits. In the original cleaned data, the majority of clients did not subscribe to a term deposit, so the number of "no" cases is much larger than the number of "yes" cases. I randomly downsample the majority class to match the number of "yes" observations, producing a balanced dataset, and then randomly split this balanced data into 70% training and 30% testing sets.

Code for task6

```
table(bank_final$y)


set.seed(123)


yes_index <- which(bank_final$y == "yes")
no_index <- which(bank_final$y == "no")


n_yes <- length(yes_index)
n_no <- length(no_index)


n_yes
n_no


sampled_no_index <- sample(no_index, n_yes)
```

```
balanced_index <- c(yes_index, sampled_no_index)
bank_balanced <- bank_final[balanced_index, ]

bank_balanced$y <- droplevels(bank_balanced$y)

table(bank_balanced$y)

set.seed(123)

n_bal <- nrow(bank_balanced)
train_size <- floor(0.7 * n_bal)

train_indices <- sample(seq_len(n_bal), size = train_size)

train_data <- bank_balanced[train_indices, ]
test_data <- bank_balanced[-train_indices, ]

dim_train <- dim(train_data)
dim_test <- dim(test_data)

dim_train
dim_test
table(train_data$y)
table(test_data$y)
```

Sample output of your code for task6

```
> table(bank_balanced$y)

  no  yes
5037 5037
> dim_train
[1] 7051   20
> dim_test
[1] 3023   20
> table(train_data$y)

  no  yes
3504 3547
> table(test_data$y)

  no  yes
1533 1490
```

Code description of task6: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I first examined the class distribution of the cleaned dataset bank_final using table(bank_final$y), which confirmed that “no” cases greatly outnumber “yes” cases. To balance the classes, I identified the indices of all “yes” and “no” observations, stored their counts in n_yes and n_no, and then used sample to randomly select exactly n_yes indices from the majority “no” group. Combining all “yes” indices with the sampled “no” indices created bank_balanced, a dataset with equal numbers of positive and negative outcomes, and I removed unused factor levels from y using droplevels. Next, I set a random seed for reproducibility and drew a 70% random sample of row indices from bank_balanced to define train_data, with the remaining 30% forming test_data. The printed dimensions and class frequency tables for the training and testing sets show that both subsets are of the expected size and retain the balanced distribution between “yes” and “no” clients.

Title of the task7

Description of the task7: which task you are going to solve here and how you will solve it.

In this task, I compute descriptive statistics for the main numerical variables of the bank marketing dataset separately for clients who subscribed to a term deposit (y = "yes") and those who did not (y = "no"). By comparing group-wise means, medians, and standard deviations for age, account balance, call duration, campaign contacts, days since last contact, and number of previous contacts, I can understand how these characteristics differ between positive and negative outcomes.

Code for task7

```
numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")


stats_mean_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = mean
)


stats_median_by_y <- aggregate(
  bank_final[, numeric_cols],
```

```
  by = list(y = bank_final$y),
  FUN = median
)

stats_sd_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = sd
)

stats_mean_by_y
stats_median_by_y
stats_sd_by_y
```

Sample output of your code for task7

```
> stats_mean_by_y
    y      age   balance duration campaign    pdays  previous
1  no 40.69095  887.5784 209.6124 2.840815 36.63867 0.5063423
2 yes 41.48481 1217.7727 425.5193 2.144332 68.52710 1.1772881
> stats_median_by_y
    y age balance duration campaign pdays previous
1  no  39     414      164        2    -1        0
2 yes  38     718      431        2    -1        0
> stats_sd_by_y
    y      age  balance duration campaign     pdays previous
1  no 10.09277 1152.933 157.5302 3.175662  97.02559 2.284156
2 yes 13.30260 1241.561 193.0821 1.927399 118.90739 2.575033
```

Code description of task7: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I defined a vector of key numerical attributes (age, balance, duration, campaign, pdays, and previous) and used the aggregate function with y as the grouping factor to calculate mean, median, and standard deviation for each variable within the “yes” and “no” classes. The resulting tables show, for example, whether successful clients tend to have longer call durations, higher or lower balances, or different campaign contact patterns compared to clients who did not subscribe. Comparing these statistics helps identify which features are most strongly associated with the subscription outcome and provides a quantitative summary of differences between the two groups.

Title of the task8

Description of the task8: which task you are going to solve here and how you will solve it.

In this task, I compare the average duration of marketing calls between two categories of the housing loan variable: clients with a housing loan (housing = "yes") and clients without a

housing loan (housing = "no"). This addresses the requirement to compare the mean of a numerical variable across two distinct categories of a categorical attribute in the dataset.

Code for task8

```
mean_duration_by_housing <- aggregate(
  duration ~ housing,
  data = bank_final,
  FUN = mean
)

mean_duration_by_housing

duration_ttest <- t.test(duration ~ housing, data = bank_final)
duration_ttest
```

Sample output of your code for task8

```
> mean_duration_by_housing
  housing duration
1      no 233.8771
2     yes 235.3426
>
> duration_ttest <- t.test(duration ~ housing, data = bank_final)
> duration_ttest

        Welch Two Sample t-test

data:  duration by housing
t = -0.86031, df = 41066, p-value = 0.3896
alternative hypothesis: true difference in means between group no and group yes is not equal to 0
95 percent confidence interval:
 -4.804178  1.873253
sample estimates:
 mean in group no mean in group yes
         233.8771          235.3426
```

Code description of task8: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I used aggregate(duration ~ housing, ...) to calculate the mean call duration separately for clients who do and do not have a housing loan, obtaining two group means that can be directly compared. To formally test whether the difference in average duration is statistically significant, I applied a two-sample t-test using t.test(duration ~ housing, data = bank_final). The test output reports the estimated mean difference, confidence interval, and p-value; together with the group means, this shows whether the bank typically spends more time on calls to clients with or without housing loans and whether the observed difference is likely to be due to chance. This satisfies the project requirement to compare average values of a numerical variable across two categories of a categorical variable.

Title of the task9

Description of the task9: which task you are going to solve here and how you will solve it.

In this task, I examine and compare the variability of the account balance variable across the different age groups created earlier (young, middle, mature, and senior). This addresses the requirement to analyze measures of spread—such as interquartile range, standard deviation, variance, and range—for a numerical attribute across multiple categories of a categorical variable.

Code for task9

```
 balance_IQR_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = IQR
)

balance_sd_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = sd
)

balance_var_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = var
)

balance_range_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = function(x) max(x) - min(x)
)
```

balance_IQR_by_agegroup

balance_sd_by_agegroup

balance_var_by_agegroup

balance_range_by_agegroup

Sample output of your code for task9

```
> balance_IQR_by_agegroup
  age_group balance
1     young   959.0
2    middle  1271.0
3    mature  1635.0
4    senior  2612.5
> balance_sd_by_agegroup
  age_group  balance
1     young 1051.148
2    middle 1143.718
3    mature 1229.752
4    senior 1293.089
> balance_var_by_agegroup
  age_group balance
1     young 1104913
2    middle 1308091
3    mature 1512289
4    senior 1672079
> balance_range_by_agegroup
  age_group balance
1     young    5376
2    middle    5376
3    mature    5376
4    senior    3587
```

Code description of task9: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

Using the aggregate function with age_group as the grouping factor, I computed the interquartile range, standard deviation, variance, and range of account balance for each age category. The IQR reflects the spread of the middle 50% of balances, the standard deviation and variance capture overall dispersion around the mean, and the range summarizes the distance between minimum and maximum balances in each group. Comparing these values across the four age groups reveals which segment of clients exhibits the most volatile or concentrated balances—for example, whether seniors have more widely dispersed balances than younger clients. This helps the bank understand which age segments show greater financial variability and may require different marketing or risk strategies.

Title of the task10

Description of the task10: which task you are going to solve here and how you will solve it.

In this task, I present an overall descriptive summary of the final cleaned dataset before performing any advanced modeling. After handling missing and unknown values, outliers, invalid records, duplicates, and class imbalance, it is important to verify the central tendencies and distributions of key variables and confirm that the data are consistent with expectations from the original dataset description.

Code for task10

```
overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
                                                  "campaign", "pdays", "previous")])
overall_summary_numeric
```

```
overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
                                                      "default", "housing", "loan",
                                                      "contact", "month", "poutcome", "y")])
overall_summary_categorical
```

Sample output of your code for task10

```
> overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
+                                                    "campaign", "pdays", "previous")])
> overall_summary_numeric
      age           balance           duration        campaign         pdays           previous
 Min.   :18.00   Min.   :-1945.0   Min.   :  1.0   Min.   : 1.00   Min.   : -1.00   Min.   :  0.0000
 1st Qu.:33.00   1st Qu.:   71.0   1st Qu.:103.0   1st Qu.: 1.00   1st Qu.: -1.00   1st Qu.:  0.0000
 Median :39.00   Median :  443.0   Median :180.0   Median : 2.00   Median : -1.00   Median :  0.0000
 Mean   :40.78   Mean   :  925.9   Mean   :234.7   Mean   : 2.76   Mean   : 40.34   Mean   :  0.5843
 3rd Qu.:48.00   3rd Qu.: 1415.0   3rd Qu.:318.0   3rd Qu.: 3.00   3rd Qu.: -1.00   3rd Qu.:  0.0000
 Max.   :95.00   Max.   : 3431.0   Max.   :640.5   Max.   :58.00   Max.   :871.00   Max.   :275.0000
>
> overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
+                                                        "default", "housing", "loan",
+                                                        "contact", "month", "poutcome", "y")])
> overall_summary_categorical
          job           marital          education     default     housing      loan            contact
 blue-collar:9439   divorced: 5038   primary  : 6850   no :42568   no :19043   no :36241   cellular :28293
 management :9215   married :26051   secondary:23200   yes:  783   yes:24308   yes: 7110   telephone: 2716
 technician :7354   single  :12262   tertiary :13301                                       unknown  :12342
 admin.     :5000                    unknown  :    0
 services   :4004
 retired    :2145
 (other)    :6194
     month          poutcome       y
 may    :13208   failure: 4724   no :38314
 jul    : 6621   other  : 1775   yes: 5037
 aug    : 6069   success: 1430
 jun    : 5026   unknown:35422
 nov    : 3851
 apr    : 2822
 (other): 5754
```

Code description of task10: elaborate the main points of your code that is used to solve this particular problem and discuss the output.

I used the summary function on the numerical variables to obtain their minimum, quartiles, median, mean, and maximum values in the final cleaned dataset bank_final. These summaries confirm that age values are within a realistic adult range, balances and call durations are positive, and campaign and previous contact counts are nonnegative after filtering. I also summarized the categorical attributes, which shows the frequency distribution of job types, marital status, education levels, loan statuses, contact methods, months, and campaign outcomes. Together, these summaries provide a compact but comprehensive view of the cleaned dataset and confirm that the data are ready for interpretation and potential modeling steps.

Project Code

Give the entire code of your project here.

```
bank <- read.csv("bank-full.csv", sep = ";", header = TRUE)

dim(bank)
names(bank)
str(bank)
summary(bank)

factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")

bank[factor_cols] <- lapply(bank[factor_cols], factor)

str(bank)

table(bank$y)
prop.table(table(bank$y))

na_counts <- colSums(is.na(bank))
na_counts

factor_cols <- c("job", "marital", "education", "default",
                 "housing", "loan", "contact", "month",
                 "poutcome", "y")

unknown_counts <- sapply(bank[factor_cols], function(x) sum(x == "unknown"))
unknown_counts
```

```
bank_clean <- bank

bank_clean <- subset(bank_clean, education != "unknown")

job_mode <- names(sort(table(bank_clean$job), decreasing = TRUE))[1]
job_mode

bank_clean$job[bank_clean$job == "unknown"] <- job_mode

sapply(bank_clean[factor_cols], function(x) sum(x == "unknown"))

dim(bank_clean)

numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

summary(bank_clean[, numeric_cols])

balance_q1 <- quantile(bank_clean$balance, 0.25)
balance_q3 <- quantile(bank_clean$balance, 0.75)
balance_iqr <- balance_q3 - balance_q1
balance_lower <- balance_q1 - 1.5 * balance_iqr
balance_upper <- balance_q3 + 1.5 * balance_iqr

duration_q1 <- quantile(bank_clean$duration, 0.25)
duration_q3 <- quantile(bank_clean$duration, 0.75)
duration_iqr <- duration_q3 - duration_q1
duration_lower <- duration_q1 - 1.5 * duration_iqr
duration_upper <- duration_q3 + 1.5 * duration_iqr
```

```
balance_outliers_before <- sum(bank_clean$balance < balance_lower | bank_clean$balance
> balance_upper)

duration_outliers_before <- sum(bank_clean$duration < duration_lower |
bank_clean$duration > duration_upper)


bank_clean2 <- bank_clean

bank_clean2$balance[bank_clean2$balance < balance_lower] <- balance_lower

bank_clean2$balance[bank_clean2$balance > balance_upper] <- balance_upper


bank_clean2$duration[bank_clean2$duration < duration_lower] <- duration_lower

bank_clean2$duration[bank_clean2$duration > duration_upper] <- duration_upper


balance_outliers_after <- sum(bank_clean2$balance < balance_lower | bank_clean2$balance
> balance_upper)

duration_outliers_after <- sum(bank_clean2$duration < duration_lower |
bank_clean2$duration > duration_upper)


summary(bank_clean2[, c("balance", "duration")])


bank_work <- bank_clean2


bank_work$age_group <- cut(

  bank_work$age,

  breaks = c(0, 30, 45, 60, Inf),

  labels = c("young", "middle", "mature", "senior"),

  right = TRUE

)


housing_num <- ifelse(bank_work$housing == "yes", 1, 0)

bank_work$housing_num <- housing_num


duration_min <- min(bank_work$duration)
```

```
duration_max <- max(bank_work$duration)

bank_work$duration_norm <- (bank_work$duration - duration_min) / (duration_max -
duration_min)


str(bank_work[, c("age", "age_group", "housing", "housing_num", "duration",
"duration_norm")])

summary(bank_work[, c("age_group", "housing_num", "duration_norm")])


bank_nodup <- bank_work



dup_flags <- duplicated(bank_nodup)

dup_count <- sum(dup_flags)


dim_before_dup <- dim(bank_nodup)


bank_nodup <- bank_nodup[!dup_flags, ]


dim_after_dup <- dim(bank_nodup)


dup_count

dim_before_dup

dim_after_dup



bank_final <- bank_nodup


invalid_age <- sum(bank_final$age < 18 | bank_final$age > 95)

invalid_duration <- sum(bank_final$duration <= 0)

invalid_campaign <- sum(bank_final$campaign < 1)
```

```
invalid_age
invalid_duration
invalid_campaign

bank_final <- subset(
  bank_final,
  age >= 18 & age <= 95 &
    duration > 0 &
    campaign >= 1
)

dim_bank_final <- dim(bank_final)
summary(bank_final[, c("age", "duration", "campaign")])



table(bank_final$y)

set.seed(123)

yes_index <- which(bank_final$y == "yes")
no_index <- which(bank_final$y == "no")

n_yes <- length(yes_index)
n_no <- length(no_index)

n_yes
n_no

sampled_no_index <- sample(no_index, n_yes)
```

```
balanced_index <- c(yes_index, sampled_no_index)
bank_balanced <- bank_final[balanced_index, ]

bank_balanced$y <- droplevels(bank_balanced$y)

table(bank_balanced$y)

set.seed(123)

n_bal <- nrow(bank_balanced)
train_size <- floor(0.7 * n_bal)

train_indices <- sample(seq_len(n_bal), size = train_size)

train_data <- bank_balanced[train_indices, ]
test_data <- bank_balanced[-train_indices, ]

dim_train <- dim(train_data)
dim_test <- dim(test_data)

dim_train
dim_test
table(train_data$y)
table(test_data$y)


numeric_cols <- c("age", "balance", "duration", "campaign", "pdays", "previous")

stats_mean_by_y <- aggregate(
  bank_final[, numeric_cols],
```

```
  by = list(y = bank_final$y),
  FUN = mean
)

stats_median_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = median
)

stats_sd_by_y <- aggregate(
  bank_final[, numeric_cols],
  by = list(y = bank_final$y),
  FUN = sd
)

stats_mean_by_y
stats_median_by_y
stats_sd_by_y


mean_duration_by_housing <- aggregate(
  duration ~ housing,
  data = bank_final,
  FUN = mean
)

mean_duration_by_housing

duration_ttest <- t.test(duration ~ housing, data = bank_final)
```

```
duration_ttest



balance_IQR_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = IQR
)

balance_sd_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = sd
)

balance_var_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = var
)

balance_range_by_agegroup <- aggregate(
  balance ~ age_group,
  data = bank_final,
  FUN = function(x) max(x) - min(x)
)

balance_IQR_by_agegroup
balance_sd_by_agegroup
```

```
balance_var_by_agegroup
balance_range_by_agegroup


overall_summary_numeric <- summary(bank_final[, c("age", "balance", "duration",
                                                  "campaign", "pdays", "previous")])
overall_summary_numeric

overall_summary_categorical <- summary(bank_final[, c("job", "marital", "education",
                                                      "default", "housing", "loan",
                                                      "contact", "month", "poutcome", "y")])
overall_summary_categorical
```